NOE+ / NOESHADER

Engineering implementation prompt | NoeEngine v0.1.1

NoeEngine startup : probe API, GPU capabilities, driver

NoeShader / Forward+ (Vulkan)

High-capability rendering path

Noe+ Lite / Compatibility

Older supported OpenGL GPUs

Quality policy : Auto / Lite / Balanced / High / Ultra

Purpose: Extend the existing Godot Forward+ renderer, present it as NoeShader, and introduce adaptive Noe+ graphics quality while preserving a first-class low-end Compatibility renderer.

Repository: EMN90909/NoeEngine (main). Source baseline documented as Godot 4.0.5 RC / NoeEngine 0.1.0-dev. This is an implementation specification, not evidence that work has shipped.

Non-negotiable: Preserve scene/project compatibility, existing source licensing and attributions, startup safety, low-resource editor usability, and stable frame pacing.

Deliverables: audited source changes, Auto/Lite/Balanced/High/Ultra presets, GPU feature probing, safe renderer choice, tests, benchmark scenes, screenshots, documentation and validated Linux build.

SYSTEM ARCHITECTURE

One product, two existing rendering paths

NoeEngine startup : probe API, GPU capabilities, driver

NoeShader / Forward+ (Vulkan)

High-capability rendering path

Noe+ Lite / Compatibility

Older supported OpenGL GPUs

Quality policy : Auto / Lite / Balanced / High / Ultra

NOE+ is the adaptive graphics family. NoeShader is the user-facing label for the existing Forward+ clustered renderer. Noe+ Lite uses the existing GLES3/OpenGL Compatibility renderer.

Detect API/driver/device capabilities at startup. Select a supported backend before rendering; choose a separate quality preset. A manual override must be permitted and verified before application.

Adapt resolution and expensive optional effects at runtime within the already chosen backend. Do not attempt a mid-frame Vulkan/OpenGL backend swap.

High fidelity is an aspiration to measure. A label change by itself does not increase graphical quality.

FILE MAP / CHANGE TARGETS

Source-verified paths and responsibilities

main/main.cpp : trace rendering_method / rendering_driver CLI and startup selection; integrate feature-probe and safe fallback without hijacking explicit overrides.

editor/project_manager.cpp : label Forward+ as NoeShader and show readable renderer guidance while retaining serialized internal IDs.

servers/rendering/renderer_rd/forward_clustered/render_forward_clustered.cpp (+ header): core clustered forward render passes, MSAA and attachments; benchmark before altering scheduling or buffers.

servers/rendering/renderer_rd/renderer_scene_render_rd.cpp (+ header): shared lighting, shadows, sky, GI and environment integration. Guard options by supported capabilities.

servers/rendering/renderer_rd/renderer_compositor_rd.cpp : confirmed shader cache setup; profile caching, invalidation and GPU rendering lifecycle.

servers/rendering/renderer_rd/effects/fsr.cpp : existing FSR compute pipeline and fallback; investigate connecting quality controls and dynamic scaling.

servers/rendering/renderer_rd/storage_rd/render_scene_buffers_rd.cpp : scaling/sharpness and render targets; instrument memory and viewport behavior.

servers/rendering/renderer_rd/forward_mobile/render_forward_mobile.cpp : preserve mobile Vulkan path; do not mix with OpenGL compatibility.

drivers/gles3/rasterizer_gles3.cpp : Compatibility fallback validation. Review GLES3 limitations for Lite presets.

platform/linuxbsd/detect.py, build.sh, BUILD_PRODUCTS.md : native toolchain and packaging checks. Inspect additional platform-specific driver initialization paths before editing.

AI ENGINEERING EXECUTION PROMPT

Copy these instructions into a coding agent

Act as a senior C++ renderer engineer maintaining a Godot 4.0.5 RC fork. Work in EMN90909/NoeEngine, branch from a known main SHA, and treat Changes/Noe+/checklist.md as the acceptance register.

Phase A (READ): trace renderer selection from CLI and project manager into RenderingServer/compositor and platform drivers. Inspect full source and shader files, not only the listed snippets. Record affected call graph and configuration keys.

Phase B (MEASURE): compile an unmodified baseline; collect Linux editor start time, CPU/RAM, GPU/driver information, frame times and screenshots for matched scenes. Flag any unmeasured assumptions.

Phase C (DESIGN): implement a small capabilities and quality-policy layer without replacing Godot Forward+. Separate backend startup choice from runtime graphic preset. Keep historical renderer IDs and CLI compatibility.

Phase D (IMPLEMENT): change only presentation strings for Forward+ naming; wire Auto/Lite/Balanced/High/Ultra with feature-gated values, manual overrides, and conservative recovery when Vulkan is unavailable.

Phase E (OPTIMIZE): use profiler evidence to target specific shader cache stalls, GI/fog/reflection costs, render buffer allocations and scaling behavior. Avoid speculative large renderer rewrites.

Phase F (VERIFY): unit, integration and graphics smoke checks; compare screenshots and frame-time distributions; record cross-platform coverage. Fix regressions or explicitly document blocking failures.

Phase G (DELIVER): provide changed files, rationale, commits, metrics, known limitations and completed checklist entries. No unsupported ’production-ready’ claim and no CI green claim without a real compiled/run test.

QUALITY POLICY

Adaptive settings must be capabilities-driven

Auto: probe supported API features and available resources; use conservative defaults, optional bounded calibration, and persist a user override. Hardware model-name lookup alone is insufficient.

Lite: Compatibility where required; prioritize memory use, simplified shadows and effects, 3D render scaling if actually supported, and editor responsiveness.

Balanced: midrange features on an appropriate renderer. Keep volumetrics, GI and reflection costs controlled and profile shader complexity.

High: higher quality shadows, probe/reflection selection, GI and postprocessing only when device capabilities and frame budget permit.

Ultra: quality-first settings on capable hardware; do not automatically require ray tracing or features the old engine does not provide.

Adaptive controller: sample smoothed frame times, use hysteresis, cooldown, min/max render scale and stability guardrails. Never alter active rendering backend in a frame.

Failure behavior: preserve explicit overrides when supported; when unsupported, log reason and offer safe fallback rather than silently corrupting settings.

TEST MATRIX

Evidence-based release validation

Linux low-end: Celeron 1005M / 6 GB / older Intel integrated GPU on antiX is a desired test target, not a confirmed compatible configuration. Verify OpenGL requirements, driver versions, editor startup and 1366x768 usability.

Linux modern: run Vulkan-capable Intel/AMD/NVIDIA hardware with multiple driver versions. Test cold/warm shader caches, renderer picker, compatibility fallback and GPU feature reporting.

Windows and macOS: test separately. Never infer driver behavior from a Linux pass. MoltenVK and native OpenGL differences require platform-specific validation.

Scenes: blank editor; simple 2D; basic 3D; clustered lights stress; shadows; GI/fog/reflections; transparencies/particles; resize and multi-viewports; scene reopen.

Metrics: startup cold/warm duration, peak/resident RAM, GPU memory where available, p50/p95/p99 frame time, frame stalls, compile-time, artifact size and draw calls.

Regressions: compare screenshots at equivalent settings. Test unsupported Vulkan, missing extensions, shader cache corruption, OOM conditions and manual rendering overrides.

Release gate: reproducible native build plus actual editor/game execution, no critical compatibility regressions, preset serialization tests and documented platform coverage.

RISKS / NON-GOALS

Protect the fork from expensive mistakes

Risk HIGH: Godot 4.0.5 RC is an old baseline. Compare modern upstream fixes and backport selectively only with compatibility testing. Do not mass-merge unrelated engine rewrites.

Risk HIGH: repository build documentation explicitly says native engine build is not yet verified. Source presence does not equal runtime support.

Risk MEDIUM: Forward+ buffer attachments and GI/reflection features may consume considerable memory. Change lifetime and formats only after instrumenting allocations.

Risk MEDIUM: documentation describing editor features can lag actual source. Preserve current Noe-specific account/update manager and Noqeri editor integrations.

Non-goal in v0.1.1: custom ray-tracing system, custom virtual geometry, new standalone renderer or parity with Unreal Engine 5.

Non-goal: NoeEngine Cloud GPU-rendering service. Remote batch rendering is a separate paid service, not part of the local NoeShader Vulkan/OpenGL backend decision.

Legal: Godot-derived MIT source and attribution remain intact. Do not indiscriminately rename internal API names, filename headers, source copyrights or license notices.

ACCEPTANCE AND HANDOFF

What done actually means

Backend: Auto startup selects supported NoeShader/Compatibility path, respects safe user overrides, and explains fallback. Historical Godot project renderer IDs remain functional.

Quality: Auto plus four documented presets, manual controls and support gating work. Adaptive 3D scaling does not degrade editor UI or repeatedly oscillate.

Performance: measured results demonstrate improvements or trade-offs against the same engine baseline on identical hardware/scenes. Include regressions transparently.

Compatibility: GDScript, native Noqeri bridge, editor accounts, project manager, export templates, project reopen and packaged runtime remain functional.

Testing: native Linux binary actually compiled and launched. Other platforms pass independently or are visibly marked unverified.

Reporting: complete Changes/Noe+/checklist.md, record source files touched, tests, benchmark methodology, screenshots, risks, fallback limitations and follow-up scope.

Suggested phases: (1) baseline & renderer choice (2) presets & UI (3) profiling-based optimizations (4) broad validation. Avoid committing everything as one opaque patch.